Software Design Document

February 27th, 2026

KIMinder360 Team
Dylan Hyer (deh277), Nyle Huntley (nah353),
Cole Bishop (cdb499), Mayanna John (mrj335)
Morgan W. Boatman (Sponsor)
Md Nazmul Hossain (Mentor)

Version 2.3

Table Of Contents

Introduction

KIMinder360 is a mobile application designed to transform situational awareness training from a manual, time-consuming process into an automated, scalable, and easy-to-use experience. Modern society is increasingly struggling with a lack of situational awareness, with many people unaware of their surroundings while using their phones or navigating public spaces. Despite this trend, attentiveness and preparedness remain crucial skills for safely and effectively navigating today's world. However, most wellness and self-improvement applications lack tools specifically aimed at training users to be more aware of their surroundings. KIMinder360 addresses this gap by delivering short, real-world situational awareness training challenges called directives to users through their phones using a fully automated process.

The planned solution is an offline-first mobile application that delivers randomized situational awareness directives to users via push notifications to users during user-defined “active windows.” Users can customize directive categories, engagement level, active hours, and directive frequency, while the system tracks progress locally and optionally synchronizes with a cloud database to support community features. At its core, the application must satisfy the following domain-level user requirements: automated directive delivery, strong user customization and control, local management of directives and user preferences, implementation of safety and privacy safeguards, and support for future scalability and growth.

Key functionalities supporting these core requirements include automated push notifications, a local database of directives, category selection, and active hour scheduling. Performance requirements focus on system responsiveness and speed to meet certain launch-time and load-time expectations. Usability requirements emphasize ease of use for new users, ensuring a short learning curve and the ability to complete tasks within a reasonable amount of time. Secure data storage is required to support offline functionality, preserve user preferences, and protect user data. Additional performance constraints require background tasks to stay within defined limits for memory, CPU, and battery usage. Environmental requirements include hardware and platform constraints, such as targeting Android 10 or higher and requesting appropriate notification permissions for full functionality. Software and development constraints specify the use of Kotlin as the primary programming language, Room for database management, and the Android Notification Manager API for handling notifications. Finally, the application must adhere to ethical standards and meet the distribution requirements for distribution on the Google Play Store.

Implementation Overview

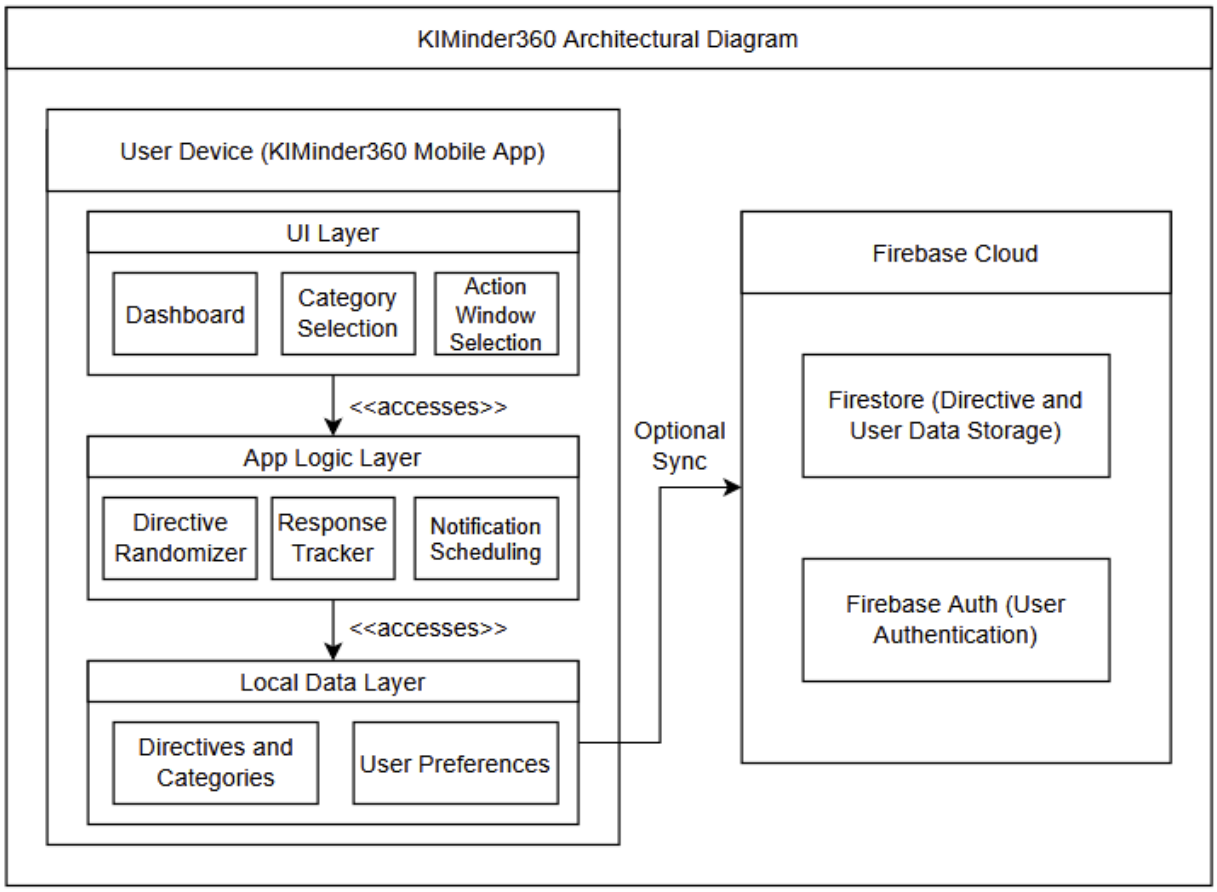
KIMinder360 is implemented as a native Android application targeting modern smartphones and tablets. The overall design is shaped by several constraints, including a mobile-only platform, a four month development timeline, an offline-first requirement, and expectations for future growth and scalability. Because there is no existing software system to integrate with, the application is being developed from the ground up using Android Studio.

The solution vision is to encapsulate the situational awareness training approach of our client, Morgan Boatman, into a self-sustaining app that automates the training delivery process while requiring minimal external input. To support this vision, the system adopts a layered architecture built on Android's recommended Model-View-ViewModel (MVVM) pattern. Kotlin serves as the primary programming language, while Room provides local database functionality. Android's built-in NotificationManager and WorkManager are used to manage directive delivery and scheduling in a reliable manner. Firebase is identified as a potential, though not yet implemented, cloud-based synchronization option that may be introduced if required to support distributing directive updates to all users or enabling community-based interactions.

At a high level, the system is composed of four conceptual layers: the user interface layer, implemented using Jetpack Compose; the application logic layer, written in Kotlin and organized around ViewModels; the local data layer, managed through a Room database; and an optional cloud integration layer with Firebase. This approach separates concerns, improves testability and maintainability, and enables future features to be added while preserving the stability of the core offline functionality.

Architectural Overview

The KIMinder360 architecture follows a client-centric, layered design with optional client-to-cloud synchronization. The major components of the system include the UI Layer, the App Logic Layer, Local Data Layer, Notification system dependencies, and an optional cloud integration layer.



[Figure 1: Architecture Diagram showing the general layout for the KIMinder360 App]

This system is implemented as a single mobile client (Monolithic app) rather than as a distributed service architecture. This approach was selected to align with the project's constraints and usage expectations: Core functionality is required to operate entirely offline, the initial user base is expected to be small to moderate, and a monolithic mobile client design reduces deployment, operational complexity, and ongoing maintenance costs.

Client-server separation exists only at the data synchronization boundary. All core logic and data related to directive selection, scheduling, and progress tracking are executed and stored locally on the device. Optional Firebase synchronization may be introduced to support features such as community directives and leaderboard functionality;

however, the application remains fully usable without these features enabled. Within the client, communication between the user interface and ViewModels is synchronous, while interactions involving schedulers, background workers, and the notification system are handled asynchronously. External dependencies are limited to Android operating system services for notifications and scheduling, as well as Firebase APIs when connected to the internet.

Component-Level Design

UI Layer

The UI Layer functions as the presentation boundary between the user and the system. Its primary responsibility is to render application state, collect user input, and immediately reflect configuration changes while avoiding direct implementation of business logic or data persistence. This layer presents menus and interfaces that allow users to adjust categories, active hours, directive frequency, and directive interactions.

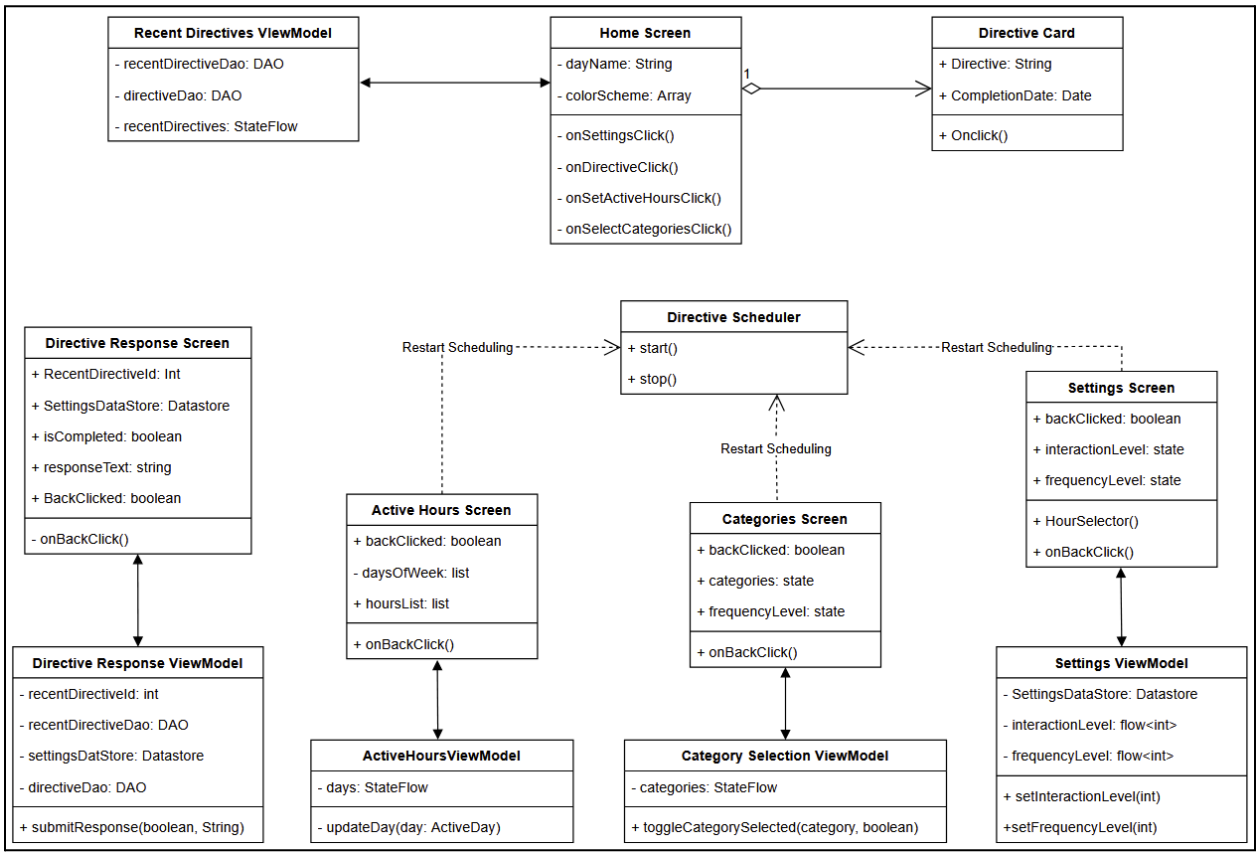
It observes application state through LiveData and StateFlow streams exposed by ViewModels and reacts to updates in real time to ensure a responsive user experience. The UI Layer does not perform scheduling, random selection, or database operations. Instead, all logic related actions are delegated to the App Logic Layer through ViewModels. This separation prevents tight coupling between presentation and business logic and ensures the interface remains focused on rendering and user interaction.

Communication follows a unidirectional and traceable data flow model: the UI observes ViewModel state streams and emits user-driven events, such as preference updates or configuration changes, back to the App Logic Layer.

Internal Structure and Components

- Main Menu: Serves as the primary navigation hub and entry point to core application features.
- Directive History Fragment: Displays previously sent directives, completion status, and progress over time.
- Settings Fragment: Provides centralized access to user customization options and preferences.
- Category Selection Fragment: Allows users to enable or disable specific directive categories.

- Active Hours Selection Fragment: Enables users to configure time windows during which directives may be delivered.
- Frequency Selection Fragment: Allows users to define how frequently directives should be scheduled within active hours.



[Figure 2: Class Diagram for User Interface Layer]

App Logic Layer

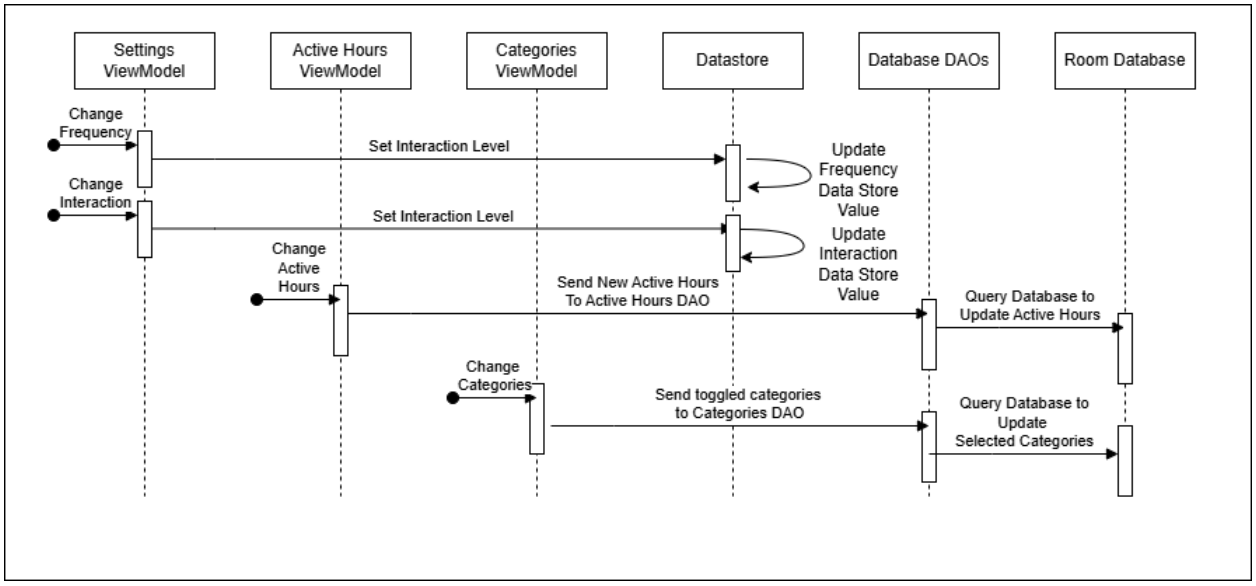
The App Logic Layer serves as the core coordinator of the application. It enforces application rules, processes user preferences, and orchestrates communication between the UI Layer, Local Data Layer, and the Notification System. This layer is responsible for enforcing application constraints, randomly selecting directives from active categories, scheduling notifications within user-defined active windows, and maintaining the current application state exposed to the UI.

Directive delivery logic ensures that notifications respect configured frequency, interaction levels, and time restrictions. The App Logic Layer does not render UI elements or directly manipulate raw database tables. Instead, it consumes structured data provided by the Local Data Layer and produces processed outputs such as scheduled notification requests and UI state updates.

Inputs to this layer include user preferences, database query results, and system time. Outputs include selected directives, scheduled background tasks for notification delivery, and observable state streams for the UI Layer.

Key Modules

- Random Directive Selector: Filters directives based on active categories and randomly selects eligible directives using controlled randomization logic.
- Notification Scheduler: Determines valid delivery times within active hours and queues background tasks accordingly.
- Preferences ViewModel: Acts as the intermediary between UI and data sources, exposing observable state streams and handling user configuration logic.



[Figure 3: Class Diagram for App Logic Layer]

Local Data Layer

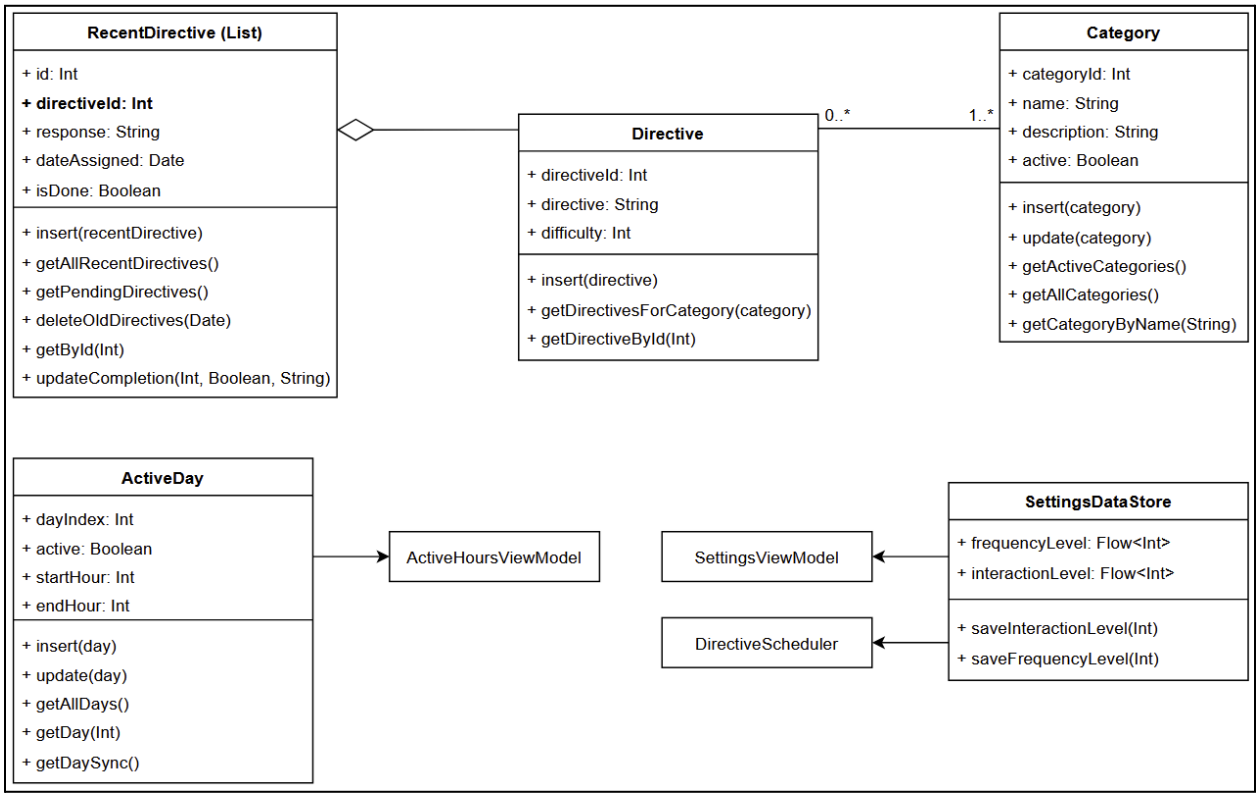
The Local Data Layer serves as the persistence foundation of the application. It manages structured storage and retrieval of directives, user preferences, and historical data while supporting fully offline functionality. This layer is responsible for executing create, read, update, and delete (CRUD) operations and maintaining long-term data integrity across application restarts.

The Local Data Layer stores directive content, category configurations, active hours, user preferences, and directive completion history. It does not contain business logic or handle UI concerns. Instead, it provides reliable data streams and transactional storage mechanisms to higher layers of the system. ViewModels observe these data streams to

reflect current state in the UI, while the App Logic Layer submits updates to ensure consistent data persistence.

Core Entities

- Directives Entity: Stores directive content and associated metadata for filtering and selection.
- Categories Entity: Defines directive groupings and maintains activation states for user-selected categories.
- RecentDirectives Entity: Tracks directive completion records, user responses, and associated timestamps.
- ActiveDays Entity: Stores scheduling constraints defined by the user.
- Settings DataStore: Persists configuration data such as directive frequency, interaction level, and other customization settings.



[Figure 4: Class Diagram for Local Data Layer]

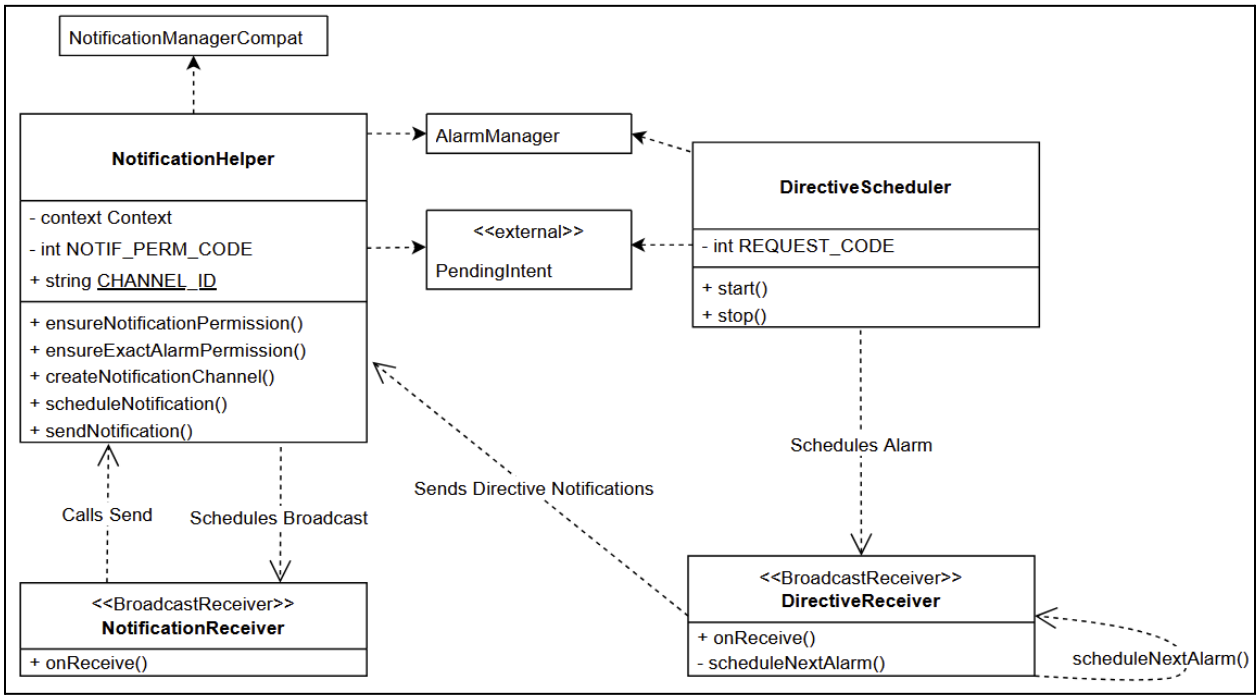
Notification System

The Notification System is a subsystem of the App Logic Layer responsible for time-based directive delivery and background execution of scheduled tasks. Its primary role is to deliver directives at scheduled times, enforce active window constraints, and ensure configured frequency limits are respected. It also handles user interaction with

notifications by launching the appropriate directive detail or response screen when a notification is selected. This subsystem does not determine which directives are selected or when they should be scheduled, as those decisions are made by the Notification Scheduler in the App Logic Layer.

Implementation Components

- Android Notification Manager: Responsible for displaying system notifications to the user.
- WorkManager: Executes deferred and background scheduling tasks reliably, including across device restarts and varying system conditions.



[Figure 5: Class Diagram for Notification System]

Optional Cloud Layer

The Optional Cloud Layer extends the application's capabilities beyond local storage by supporting community-driven content and cross-device synchronization while remaining decoupled from the offline core. This layer is planned to supply community directives, synchronize user progress across devices, and enable leaderboard functionality. It enhances scalability without introducing dependency on network availability for core features.

Firebase was selected due to its seamless Android integration, scalability, and real-time synchronization capabilities. Importantly, this layer is isolated so that the core directive

scheduling and delivery system remains fully functional offline. When enabled, it synchronizes with the Local Data Layer and provides additional directive data to the App Logic Layer without altering core scheduling logic.

Implementation Plan

Development Timeline

Phase 1 (Weeks 1-5): Foundational Database Linking and Core Logic

This phase establishes the core data models, persistence layer, and baseline application logic required for directive generation and scheduling. Work in this phase focuses on building the foundational infrastructure upon which all future features and extra functionality depends.

UI Components

- Implement the Settings Page as the central access point for user customization.
- Create the Frequency Customization Screen to allow users to specify how often directives should be delivered.
- Implement navigation between the home, settings, and customization screens.

App Logic Layer

- Implement a random category selection function to select from active categories.
- Implement a random directive selection function that retrieves a directive from a specified category while avoiding recent duplicates.
- Create a directive scheduling function that assigns a directive to a specific time within the user's defined action window.
- Begin encapsulating logic in ViewModels to enforce separation of concerns.
- Combine specific purpose functions to implement automated directive delivery using WorkManager and Android's notification APIs.
- Ensure notifications respect action windows, respect frequency limits, and pull randomized directives from enabled categories

Local Data Layer

- Define and implement room entities for directives, categories, active hours, and recent directives
- Create a local datastore for user preferences
- Implement DAOs to support CRUD operations and reactive data streams for UI.

Key Deliverables

- Functional settings and frequency configuration UI
- Working local database with linked entities
- Verified random selection and scheduling logic
- Directive response and history tracking

Phase 2 (Weeks 6-10): Added Functionality and Feature Completion

This phase focuses on completing the core directive experience by implementing improvements to our delivery algorithm, interaction with notifications, and expanded user accessible data management. The emphasis of this phase is enabling users to receive, respond to, review, and manage directives in a reliable and intuitive way while refining the overall user interface.

UI Components

- Have Directive Response Screen launched when notification is tapped.
- Display different response options based on the user's selected interaction level.
- Enhance the home screen dashboard to display recently delivered directives, send time of recent directives, and completion status of directives
- Add user controls to delete recent directives from history and view/manage custom user created directives
- Finalize dark mode support and general UI polish for readability and accessibility.

App Logic Layer

- Improve Notification Scheduling Window to not run every day if not needed for better battery life and not send recent directives to avoid repeated directives
- Implement engagement-level logic:
    1) No Engagement: notification only
    2) Minimal Engagement: Yes/No completion confirmation
    3) Full Engagement: written response input
- Implement logic for skipping, temporarily silencing, and postponing directives.

Local Data Layer

- Allow users to delete specific recent directives directly from the response screen.
- Implement CRUD support for custom user created directives, stored locally and categorized under a "Personal" category.
- Expand the local database with more preloaded directives.

Notification System

- Notification tap action should take user directly to a Directive Response Screen.

- Add logic to notification builder to allow users to respond in a notification (Directive completed yes/no or free response directive from the notification depending on their interaction level).
- Ensure notifications are cancellable, reschedulable, or postponeable.
- Validate reliability across device restarts and app background states.

Key Deliverables
- Fully automated directive notification system with complex algorithm
- Functional directive response flow based on engagement level
- Recent directive history management
- Support for creating, editing, and deleting custom directives
- Polished and accessible UI (including dark mode)

Phase 3 (Weeks 11-15): Polishing, Stretch Goals, and Deployment Preparation

This phase serves as a refinement and expansion period focused on stability, client feedback, stretch goal implementation, and preparing the application for public testing and deployment. Core functionality is considered complete at the start of this phase; work here emphasizes quality, scalability, and future growth readiness.

UI and UX Polish
- Apply bug fixes and UI refinements based on internal testing and sponsor feedback.
- Improve navigation flow, spacing, animations, and visual consistency.
- Optimize dashboard layouts and progress visualization.
- Finalize onboarding flow, including legal disclaimers, permission prompts, and introduction tutorial screen.

App Logic and Performance Optimization
- Optimize scheduling and notification logic to reduce battery and CPU usage.
- Address edge cases involving overlapping Action Windows and rapid preference changes.
- Conduct performance profiling to ensure compliance with CPU, memory, and battery requirements.
- Improve error handling, logging, and fail safe behaviors.

Stretch Goals and Optional Features
- Begin optional cloud database integration (Firebase) for community-shared directives
- Cross platform implementation (iOS Version)

- Implement infrastructure hooks for future features such as leaderboards, admin dashboard, and premium directive packs obtainable through in app purchases.
- Complete active hours refactoring to add functionality for multiple action windows in a day (Revolves around “Add” button where an action window can be added to any day with any start/end hours)
- Ensure cloud features remain isolated from core offline functionality.

Testing and Deployment

- Conduct end-to-end testing across multiple Android versions and both physical and emulated devices.
- Perform usability testing against defined performance and learning metrics.
- Prepare Google Play Store assets such as an app description, screenshots and, privacy policy/disclaimers.
- Deploy to internal testing track on Google Play Store for final validation.

Key Deliverables

- Stable, polished application meeting performance and usability requirements
- Optional cloud feature prototypes (if pursued)
- Fully tested MVP ready for public or sponsor facing release
- Google Play Store deployment for testing or launch

[Figure 6: Visual diagram for milestone roadmap based on implementation plan]

The milestone roadmap above outlines a phased development approach aligned with the system's architecture, enabling parallel implementation while preserving critical dependencies. Phase 1 establishes the foundational data models, directive selection logic, scheduling mechanisms, and core settings UI that all subsequent features rely on. Phase 2 builds upon this foundation by completing the end to end directive experience, including interactive notifications, engagement level responses, directive history management, and user created content, with UI, logic, and database enhancements progressing concurrently. Phase 3 focuses on refinement, optimization, and scalability, incorporating bug fixes, performance tuning, usability improvements, optional cloud integration, and deployment preparation. Throughout development, the offline first requirement is preserved, cloud features remain isolated from core functionality, and testing is performed incrementally to ensure stability, reliability, and readiness for release.

Technical Risks

Risk	Description	Mitigation Strategy
Android Background Restrictions	Modern Android Versions are increasingly aggressive with battery optimization which can delay or kill the work manager that we rely on for sending directives.	Utilize expedited jobs and provide a dedicated settings guide for users to exclude KIMinder360 from battery optimization to ensure high priority delivery.
Room Schema Mitigations	As we add features like personal categories and change the data we store the database schema will change. Improper migration can lead to app crashes or loss of user data.	Implement automated migrations after app deployment and maintain strict versioning control. Comprehensive testing for migration paths is needed. During development, destructive migrations will be used.
Tooling and Library Volatility	Relying on jetpack compose and specific firebase libraries means we are subject to breaking changes or depreciations in future android studio updates.	We will stick to stable library resources rather than any experimental features to ensure maximum compatibility. Encapsulate third party dependencies within wrapper classes to minimize effects if a library needs replacing.
Data Volume	The local database is designed to hold thousands of directives but as the volume of custom written responses increases, we could be looking at an eventual storage impact.	We will allow users to manually delete recent directives and set an option for them to be automatically deleted after a certain number of days.

Trade Offs

1. **Local First vs. Cloud First**
    - **Decision**: We chose a local first approach using room

- **Trade Off**: This provides superior speed, offline reliability, and privacy. However, the trade off is that if a user loses their device their progress and personal directives are lost unless there is a cloud backup.

2. **Monolithic Client vs. Modular Design**
    - **Decision**: The app is built as a single module monolith
    - **Trade Off**: This simplifies development for a small team and reduces build times. The trade off is that as the app scales, it may become difficult for multiple developers to work on features without code conflicts.

Conclusion

The KIMinder360 project is all about creating a robust, offline first, situational awareness training platform. The concept originated with our client Morgan Boatman, founder of Winter Communication LLC, who envisioned making situational awareness training easily accessible to a broad audience. A key challenge in prior efforts was the limited scalability of a manual training model. Our team's goal is to design and implement a mobile solution that automates the delivery of this training, enabling our client to scale it to a large user base and help people begin improving their situational awareness.

This Software Design Document presents a clear and structured plan for implementing the KIMinder360 mobile application in support of that goal. The design emphasizes modularity, safety, and scalability while remaining grounded in practical constraints such as mobile performance considerations and a limited development timeline. By leveraging a layered MVVM architecture, Room-based data persistence, and Android's notification framework, the proposed design supports the project's functional goals while meeting performance requirements. Deliberate design decisions, including a local-first data strategy and optional cloud integration, ensure immediate usability while preserving long-term growth potential through community-driven features.

At the time of writing, the base functionality of the app has been completed, and development efforts are focused on usability improvements and expanded user customization. Testing is currently being conducted internally, with plans to release a public beta in the near future. Internal usage has provided promising validation of the concept. The core engagement loop has proven effective in encouraging users to think critically throughout the day by presenting short, actionable challenges. The offline-first design ensures consistent accessibility, while extensive user customization allows individuals to engage with training at their own pace.

As development continues, additional refinements and feature enhancements will build upon this stable architectural foundation. We are confident that continued iteration will further strengthen the user experience and support KIMinder360’s mission of making situational awareness a practical and sustainable daily habit through accessible, engaging mobile technology.